R PROGRAMMING LAB FILE

Submitted By:
Dev Verma

Roll Number:
25/SCA/MCAN/029

INDEX

Date	Experiment Name	Signature
21st Jan	Introduction to R, R studio, data types, data structures of R	
	Write R program to create a vector of first 50 even numbers	
	Create & Display a vector of first 50 odd numbers	
	Create a vector with 12 integers and convert it into a 4x3 matrix B using matrix()	
	Write R program to add 2 vectors of Integer type and length 3	
	Create a vector with the pattern: 1, 2, ..., 19, 20, 19, ..., 1	
	Write R program to divide 2 vectors of Integer type and length 3	
	Write R program to reverse the order of a given Vector	
28th Jan	R Program: Add, Delete, and Modify Elements in a Vector	
	R Program: To generate 15 random numbers and Find Numbers Greater Than the Average.	
	Write an R program to perform the following operations on a vector: Find sum, Find mean, Find maximum and minimum values · Addition, Subtraction, Multiplication, Division.	
	Create a vector of Colours of book cover and print its class	
	Create 3 vectors comprising of Vector 1- empid(1,2,3,4) Vector 2- empname(“priyanka”,”Isha”,”rohan”,”prakash”) Vector 3- numberofemp=4 Create a list containing these 3 vectors and print it.	
04 feb	Creating the matrix of 2 rows and 3 columns Creating an array of dim(3,3,2) Creating a factor Create following dataframes df = data.frame(Name, Language, Age) Create the data frame family_info including columns as Gender, Height, Weight and Age. Create the vectors for data frame. height <- c(132,151,162,139,166,147,122) weight <- c(48,49,66,53,67,52,40) gender <- c("male","male","female","female","male","female","male")	

	Exercise: 1. a. Create a vector of first 20 numbers. b. Create a vector named ‘class_students’ with names of 10 students of 3rd Grade stored in it. c. Create a vector to store age of these 10 students. 2. Create a matrix of any four flower names as elements with number of rows as 4 and columns as 2. 3. Create an array of 5 family member names with dimensions dim = c(4,3,2). a. Create a data frame to enter employee data of a small unit of company with 7 employees. It should include employee name, ID, salary, age and qualification. b. Print the qualification column.	
11 feb	Exercise: 1. Execute all the commands as discussed above. 2. Create a matrix ‘a’ using matrix function with 5 rows and 3 columns, the elements should be filled by row. 3. Create another matrix ‘b’ using rbind function with 5 rows and 3 columns, the elements should be filled by row. 4. Create a matrix ‘c’ using cbind function with 3 rows and 5 columns, the elements should be filled by column. 5. Determine transpose of matrix ‘a’, store it as ‘d’. 6. Determine matrix r1 = a + b, r2 = a - b, r3 = a / b, r4 = b %*% c.	
	a.1 Create a dataframe • With three columns • Of five employees with details of employee id, name, salary and age • Extract data of all employees w.r.t employee name and salary • Extract all the data of first two employees • Extract employee name and age related data of 3rd and 5th employee • Add one more column detailing the department of the employee and print the revised data frame • Add the details of 3 more employees in the data frame and print the revised data frame	
	2. A. Create a data frame of 5 students with their name, age, height and weight information. B. Determine the number of rows and columns of the created data frame. 3. A. Create a data frame of 8 students of R programming (open elective class having B.Tech EEE, B.Com and MBA students) with their Roll No, Name, Program, Semester, email-id as attributes.	

	B. Get details of all the students with BTech-EEE Program. C. Get details of all the students with MBA Program. D. Determine its number of rows and columns. 4. Write an R program to create one data frame with columns as ID and AGE, second data frame with one additional column as CITY. Merge the two data frames based on the ID column and print the merged data frame. 5. Create a data frame with duplicate values and write an R script to find unique values in the data frame. 6. Write an R program to change a column name of a given data frame.	
13 feb	b. Implement R script to create a List, to access elements in the list and perform merging of the lists. b.1 Create a list containing strings, numbers, vectors and logical values.	
	b.2 Create a list • containing a vector, matrix and a list. • Give names to the elements in the list • Print the list • Access first element of the list • Access third element of the list • Access 2nd element of the list using its name instead of index • Access both 1st and 2nd element of the list	
	b.4 Create a list of 3 vectors and one numeric value. • Print a list • Print its 2nd element • Print its 2nd and 4th element • Print the content of 1st member of vector which as a whole is 2nd element of the list x[[2]][1]= ?? • Update this content i.e. x[[2]][1] = ta and print x[[2]]	
	b.5 Write a R program to find all elements of a given list that are not in another given list.	
	1. Execute all the commands as given above. 2. Write a R program to find row and column index of maximum and minimum value in a given matrix. 3. Use R to create the following two matrices and do the indicated matrix multiplication.	
25 feb 2026	1a Write a R script to print Hello World 4 times using ‘repeat’ loop provided the count is set at 2 initially.	

	1b Write a R script to print Hello While Loop 5 times using 'while' loop provided the count is set at 2 initially. 1c Write a R script to print first 4 alphabets using 'for' loop. 1d Write a R script to illustrate the usage of break statement. 1e Write a R script to illustrate the usage of 'next' statement. 1f. Using built-in functions, write a R script to • Create a sequence of numbers from 32 to 44. • Find mean of numbers from 25 to 82. • Find sum of numbers from 41 to 68.	
	1g. Write a R script to create a function to • Print squares of first 6 numbers with an argument • Print squares of first 5 numbers without an argument. • Create a function with arguments to compute a*b+c and call function by position of arguments and by names of the arguments. • Create a function to compute a*b and call a function using default arguments	
	Exercise: 1. Write R script for all the in between questions. 2. Write a program using for loop to create a multiplication table of any number. 3. Write a program to count the number of even numbers in a vector. 4. WAP to print the square root of the integers one to ten 5. Create a function to calculate raised power of any number. 6. Create a function fahrenheit_to_celsius that converts temperatures from Fahrenheit to Celsius. 7. Create a function that converts temperature from celsius to Kelvin. 8. Create a function to compute area of a circle.	
2 march	Implement a for loop to classify Sepal.Length in the iris dataset as "greater than 5" or "lesser than 5" Use seq() and rep() to generate sequences Create a vector (12,7,3,4.2,18,2,54,-21,8,-5) and compute a. mean of the vector b. mean of the vector by trimming 3 values from each end c. compute mean of vector (12,7,3,4.2,18,2,54,-21,8,-5,NA)	
	Create the vectors with following values and compute median of the vector a. (12,7,3,4.2,18,2,54,-21,8,-5) b. (2,13,8,7,15,3,9)	

	Create a vector (2,1,2,3,1,2,3,4,1,5,5,3,2,3) and compute mode of the vector.	
	Ques 5: 15 students were enrolled in a mathematics course. Their ages were (18,18,20,19,22,21,22,23,22,20,19,20,19,22,18). A) Find the mean age, median age and mode age of all the students. B) Find the median age of all students under age 22. Hint: age1=age[age<22]. C) Two more students enter in the class, age of both the students is 23. What will be the mean age, median age and mode age now?	
	Use typeof(), length(), and class() on various data types Basic plotting with plot() and hist() Implement recursive functions (e.g., factorial, Fibonacci) using R Create a list using list() and perform list slicing	
6 march	Create a data frame and perform various operations (add/remove columns, rows, access elements) Write a program to expand a data frame in R Perform merge operations on data frames Df1 and Df2 from the iris dataset Use apply(), lapply(), sapply() with lists and data frames Write an R program to plot a single point at coordinates (3, 5). Write an R program to plot multiple points using the vectors: x=(1,2,3,4,5),y=(2,4,6,8,10) Write an R program to plot points with different colors and symbols. Plot the points (2,4),(4,6),(6,8),(8,10)(2,4), (4,6), (6,8), (8,10)(2,4),(4,6),(6,8),(8,10) and add appropriate axis labels and title. Write an R program to draw a straight line passing through the points: (1,2),(2,4),(3,6),(4,8)(1,2), (2,4), (3,6), (4,8)(1,2),(2,4),(3,6),(4,8) Write an R program to plot a line graph using the vectors: x=(1,2,3,4,5)y=(5,4,3,2,1) Plot a line graph representing the equation: y=2x+1 where x is 1 to 10. Write an R program to draw a line with different line types and colors. Plot the line graph and add grid lines to the plot.	

	Implement recursive functions (e.g., factorial, Fibonacci) using R Create a list using list() and perform list slicing Create a data frame and perform various operations (add/remove columns, rows, access elements) Write a program to expand a data frame in R Perform merge operations on data frames Df1 and Df2 from the iris dataset Use apply(), lapply(), sapply() with lists and data frames Create a function to summarize each column in a data frame Use rbind() and cbind() to expand data	
8th April	Q1. productA <- c(20, 35, 40, 55, 70, 90) productB <- c(15, 25, 30, 45, 60, 80) months <- 1:6 1. Plot both products' sales on the same graph using different colors and line types. 2. Add appropriate axis labels and a title. 3. Insert a legend in the top-left corner that clearly distinguishes between Product A and Product B. Q2. You are given the following data representing the height (in cm) and weight (in kg) of 10 students: height <- c(150, 152, 155, 160, 162, 165, 168, 170, 172, 175) weight <- c(45, 47, 50, 55, 58, 60, 62, 65, 68, 70) 1. Create a scatter plot of height vs. weight. 2. Use different point symbols and colors to make the plot visually clear. 3. Add appropriate axis labels and a title. 4. Insert a legend that explains the meaning of the chosen point symbols/colors (e.g., male vs female students, or group A vs group B). Q3. A company records the quarterly sales distribution (in percentage) across four regions: sales <- c(25, 30, 20, 25) regions <- c("North", "South", "East", "West") Write R code to: 1. Create a pie chart showing the sales distribution. 2. Add labels for each region. 3. Use different colors for each slice. 4. Add a legend that explains which color corresponds to which region. Q4. You are given monthly temperature data for a city: months <- 1:12	

temperature <- c(15, 18, 22, 28, 32, 35, 34, 33, 30, 25, 20, 16)

Write R code to:

1. Plot the temperature as a line chart.
2. Add axis labels and a title.
3. Use a red dashed line (lty = 2).

Q5. A company records sales across four regions:
sales <- c(120, 150, 100, 130)
regions <- c("North", "South", "East", "West")

Write R code to:

1. Create a bar chart of sales by region.
2. Use different colors for each bar.
3. Add a legend explaining the regions.

Q6. You have exam scores of students from two different classes:
classA <- c(65, 70, 75, 80, 85, 90)
classB <- c(60, 62, 68, 72, 78, 82)

Write R code to:

1. Create a box plot comparing Class A and Class B scores.
2. Use different colors for each box.
3. Add a title and axis labels.

Q7. You are given height and weight data of 10 students:
height <- c(150, 152, 155, 160, 162, 165, 168, 170, 172, 175)
weight <- c(45, 47, 50, 55, 58, 60, 62, 65, 68, 70)

Write R code to:

1. Create a scatter plot of height vs weight.
2. Use different point symbols (pch) for male and female students.
3. Add a legend to distinguish between the two groups.

Q8. A company's quarterly sales distribution is:
sales <- c(25, 30, 20, 25)
regions <- c("North", "South", "East", "West")

Write R code to:

1. Create a pie chart showing sales distribution.

	2. Use different colors for each slice. 3. Add a legend explaining the regions.		
16th April	1. Generate Exploratory Analysis of Big Mart Data Set and Train Data Set 2. Graphically represent Big Mart Data Set using: -Univariate Analysis (Histogram, Boxplot) -Bivariate Analysis (Scatter plot, Bar plot) 3. Graphically represent Train Data Set using: Univariate and Bivariate Analysis 4. Implement a Linear Model using 100 variables in R 5. Import/export data using read.csv(), write.csv(), readxl, jsonlite 6. Perform data cleaning using na.omit(), is.na(), and tidyr package 7. Create a prediction model using lm() and evaluate with summary() and residual plots 8. To build a system in R that analyzes astrological charts and Panchang (Hindu almanac) data to predict doshas (planetary defects) and study correlations with health patterns or seasonal behaviors, aligning with IKS and modern data analytics.		
	To analyze, model, and visualize air and water pollution trends using real-time environmental datasets (e.g., AQI, water pH, turbidity, heavy metals), and predict future pollution levels using statistical and machine learning techniques in R. To analyze and predict stock price movements using historical market data and optimize investment portfolios using advanced statistical and machine learning techniques in R To use statistical analysis and machine learning in R to explore, model, and visualize gender equality across education, employment, income, and political participation, drawing from global datasets		
	Mini Project		

```
a<-20
b<-30
c<-a+b
print(c)
class(c)
#to convert the a value in integer
p<-as.integer(a)
class(p)
#help regarding if statement
help("if")
# to declare a list it will print all the command
ls()
#it will give you current working directory
getwd()
#if you want to save the file in one folder
#setwd("C:\t.prog")
#while declaring variable period(dot),letters,numbers,underscrore
typeof(a)
typeof(5)
typeof(5L)
typeof(5i)
#how to create an data structure
#VECTORS
#we declare a vector by c()
a<-c(1:20)
b<-c(1,5,7,8)
c<-c("aa","bb","cc","dd")
#create a vectors of week days
t<- c("monday","tuesday","wed","thru","fri","sat","sun")
#to print the index
t[3]
#when we want to print one or more value,number,items in vectors
```

```
y<-t[c(2,3,5)]
#in negative indexing it will remove the items and print rest of the items
z<-t[c(-2,-5)]
#to print a sequence we use sequence function starting,ending,jump
x<-seq(2,100,by=2)
rev(x)
x<-seq(1,100,by=2)
rev(x)
#add 2 vectors of integer type len 3
a<-c(2L,4L,6L)
b<-c(1L,3L,5L)
result=a+b
print(result)
#divide

a<-c(2L,4L,6L)
b<-c(1L,3L,5L)
result=a/b
print(result)
#subtraction
a<-c(2L,4L,6L)
b<-c(1L,3L,5L)
result=a-b
print(result)
#mean
a<-c(2L,4L,6L)
b<-c(1L,3L,5L)
result<-a+b
mean_value<-mean(result)
print(result)
print(mean_value)
#maximum and minimum value
```

```
a<-c(2L,4L,6L)

b<-c(1L,3L,5L)

result<-a+b

max_value<-max(result)

min_value<-min(result)

print(result)

print(max_value)

print(min_value)

#reverse the order of a vector

a<-c(1:20)

rev(a)

#delete

a<-a[-4]

#write a r prog to add,delete,modify the elements in a vector

v<-c(1:20)

x<-c(v,21)

#now i want to add the data in the specific position= after means which position i want to add
,30 which value

v<-append(v,30,after=2)

#modify

v[2]<-100

#write r prog to create vector of 15 random number . find the numbers greater than the
average.

v<-sample(1:100,15)

avg_v<-mean(v)

v<-v[v>avg_v]

#create a list

list1<-list(c(1,2,3),"abc",'abc',TRUE)

#Create a vector of colors of book cover and print class

a<-(c("ab","bc"))

class(a)

#create 3 vector employee id-> 1,2,3,4ids , second vector emp_name->priyanka,isha,rohan
and vector 3 number of emp->4.now create a list containing these 3 vectors and print them.
```

```
empid<-list(1,2,3,4)

emp_name<-list("priyanka","isha","rohan")

no.ofemp<-list(4)

print(empid)

print(emp_name)

print(no.ofemp)

#create a vector of 12 integers and convert into 4*3 matrix using a matrix function

a<-c(1:12)

b<-matrix(a,nrow = 4,ncol = 3)

b

#create a pattern 1,2.....20,19,18,.....

v<-c(1,2:19,20,19,18:1)

#apply all ds

matrixab<-matrix(c(1,2,3,4,5,6),nrow=2,ncol=3,byrow=TRUE)

#end dimension - array-2 layers

a<-array(c("red","green"),dim=c(3,3,4))

#factor -> nlevel - count number of courses ,

data<-c(1,2,3,4,5,6)

courses<-c("MCA","MCA","MCA","MCA-CS","MCA-DS","BCA","MCA-AIML","MCA-
CS","MCA-DS")

factor_courses<-factor(courses)

nlevels(factor_courses)

is.factor(factor_courses)# check yeh factor hai ki nhi

is.factor(data)#data is numeric and it is not a factor

#abc=factor(ab)

#ab<-c()

#dataframe

df1=data.frame(name,age,subject)

name=c("rohan","preeti","yashi","isha","rayan")

age=c("10,20,30,40,50")

subject=c("math","eng","hindi","sci","sst")

df1 <- data.frame(name=c("rohan","preeti","yashi","isha","rayan"), age=c(10,20,30,40,50),
subject=c("math","eng","hindi","sci","sst"))
```

```
#to print a particular column
df1$name
#Q1create of vector of first 20 numbers
#create a vector name class_students with names of 10 students of third grade stored in it.
#create a vector to store age of 10 students
# 1. Create a vector of first 20 numbers
numbers <- 1:20

# 2. Create a vector of names of 10 third grade students
class_students <- c("Aman", "Riya", "Rahul", "Neha", "Arjun",
                    "Pooja", "Karan", "Simran", "Ankit", "Priya")

# 3. Create a vector to store age of 10 students
ages <- c(8, 9, 8, 9, 8, 9, 8, 9, 8, 9)

numbers
class_students
ages



#2create a matrix any 4 flower names as elements with number of rows as 4,col =2.
matrixflower<-matrix(c("lily","sunflower","rose","whiterose"),nrow=4,ncol=2,byrow=TRUE)
#3create an array of 5 family member names with dimension 4,3,2.
a<-array(c("mummy","papa","bhai","bhen","dadi"),dim=c(4,3,2))
#4.(a)create a dataframe to enter emp data of a small unit of company with 7 employees. it
should include emp_name,id,salaray,age and qualification.
#(b)print the qualification col
emp <- data.frame(
  name = c("rohan","preeti","yashi","isha","rayan","madhur","kashish"),
  id = c(1,2,3,4,5,6,7),
  age = c(10,20,30,40,50,60,70),
  salary = c(10,20,30,40,50,60,70),
  qualification = c("12","10","bca","mca","bsc","mbbs","bds"))
```

```
print(emp$qualification)
#to print the specific row (name,id)
print(emp[1:2, ])
```

#create a dataframe of 5 employees contaning emp_id from 1 to 5 ,emp_name,salary and age .(a)now extract all the data of first 3 employees (b) extract all the data of all emp with respect to emp_name and salary.(c)extract employee name and age related to third and fifth employee.

```
# Create the data frame
emp <- data.frame(
  emp_id = 1:5,
  emp_name = c("Aman", "Riya", "Rahul", "Neha", "Arjun"),
  salary = c(25000, 30000, 28000, 32000, 27000),
  age = c(22, 25, 24, 26, 23)
)

emp
#(a) Extract all data of first 3 employees:selects rows 1 to 3 and all columns
emp[1:3, ]
#(b)Extract emp_name and salary of all employees:selects specific columns for all rows
emp[, c("emp_name", "salary")]
#(c) Extract emp_name and age of 3rd and 5th employee:Rows: 3rd and 5th
emp[c(3, 5), c("emp_name", "age")]
#Add one more column (Department) and print revised data frame
emp$department <- c("HR", "IT", "Finance", "Marketing", "Sales")

emp
#Add details of 3 more employees and print revised data frame
new_emp <- data.frame(
  emp_id = c(106, 107, 108),
  emp_name = c("Neha", "Rahul", "Priya"),
  salary = c(32000, 29000, 31000),
  age = c(29, 24, 28),
  department = c("IT", "HR", "Finance")
```

```
)

emp <- rbind(emp, new_emp)

emp

#Create a matrix ‘a’ using matrix() function with 5 rows and 3 columns, elements filled by row.
a <- matrix(1:15, nrow = 5, ncol = 3, byrow = TRUE)
a

#Create another matrix ‘b’ using rbind() with 5 rows and 3 columns, elements filled by row.
b <- rbind(
  c(1, 2, 3),
  c(4, 5, 6),
  c(7, 8, 9),
  c(10, 11, 12),
  c(13, 14, 15)
)
b

#Create a matrix ‘c’ using cbind() with 3 rows and 5 columns, elements filled by column.
c <- cbind(
  c(1, 2, 3),
  c(4, 5, 6),
  c(7, 8, 9),
  c(10, 11, 12),
  c(13, 14, 15)
)
c

#Determine transpose of matrix ‘a’, store it as ‘d’.
d <- t(a)
```

```
d

#Question 6:

#Determine matrices:

  #r1 = a + b
#r2 = a - b
#r3 = a / b
#r4 = b %*% c

r1 <- a + b
r1

r2 <- a - b
r2

r3 <- a / b
r3

r4 <- b %*% c
r4

#. Create a data frame of 5 students
students <- data.frame(
  name = c("Aman", "Riya", "Rahul", "Neha", "Arjun"),
  age = c(20, 21, 19, 22, 20),
  height = c(170, 160, 175, 165, 172),
  weight = c(65, 55, 70, 58, 68)
)

students
```

```
#B. Number of rows and columns
nrow(students)   # number of rows
ncol(students)   # number of columns
dim(students)    # gives rows and columns together

#Create data frame of 8 students
students <- data.frame(
  RollNo = 1:8,
  Name = c("Aman", "Riya", "Rahul", "Neha", "Arjun", "Priya", "Hiya", "Sneha"),
  Program = c("BTech-EEE", "MBA", "BCom", "BTech-EEE", "MBA", "BCom", "BTech-EEE",
"MBA"),
  Semester = c(3, 1, 2, 5, 3, 4, 7, 2),
  Email = c("a@gmail.com", "r@gmail.com", "ra@gmail.com",
            "n@gmail.com", "ar@gmail.com", "p@gmail.com",
            "h@gmail.com", "s@gmail.com")
)

students

#B. Get details of BTech-EEE students
students[students$Program == "BTech-EEE", ]

#C. Get details of MBA students
students[students$Program == "MBA", ]
```

#Write a R program to create one data frame with columns as ID and AGE, second data frame with one additional column as CITY. Merge the two data frames based on the ID column and print the merged data frame.

```
# Create first data frame
df1 <- data.frame(
```

```
  ID = c(1, 2, 3, 4),
  AGE = c(20, 21, 22, 23)
)

# Create second data frame
df2 <- data.frame(
  ID = c(1, 2, 3, 4),
  CITY = c("Delhi", "Mumbai", "Jaipur", "Pune")
)

# Merge the data frames using ID
merged_df <- merge(df1, df2, by = "ID")

# Print merged data frame
print(merged_df)
```

#Create a data frame with duplicate values and write an R script to find unique values in the R data frame.

```
# Create data frame with duplicate values
df <- data.frame(
  ID = c(1,2,2,3,4,4,5),
  NAME = c("A","B","B","C","D","D","E")
)

# Display original data frame
print(df)

# Find unique rows
unique_values <- unique(df)

# Print unique values
print(unique_values)
```

```
#Write an R program to change a column name of a given data frame.
# Create a data frame
df <- data.frame(
  ID = c(1,2,3),
  NAME = c("Ram","Shyam","Mohan")
)

# Print original data frame
print(df)

# Change column name
colnames(df)[2] <- "STUDENT_NAME"

# Print updated data frame
print(df)

#b. Implement an R script to create a List, access elements in the list and perform merging of lists.
# Create first list
list1 <- list(name="Ram", age=21)

# Create second list
list2 <- list(city="Delhi", marks=85)

# Access elements
print(list1$name)
print(list2$city)

# Merge lists
merged_list <- c(list1, list2)

# Print merged list
print(merged_list)
```

```
#b.1 Create a list containing strings, numbers, vectors and logical values.
# Create a list with different data types
my_list <- list(
  string = "Hello",
  number = 25,
  vector = c(10,20,30,40),
  logical = TRUE
)

# Print the list
print(my_list)

#b.2 Create a list
#Question

#Create a list:
 # containing a vector, matrix and a list
#Give names to the elements in the list
#Print the list
#Access first element of the list
#Access third element of the list
#Access 2nd element using its name instead of index
#Access both 1st and 2nd elements of the list

# Create a vector
v <- c(10,20,30)

# Create a matrix
m <- matrix(c(1,2,3,4), nrow=2)

# Create a list
```

```
l <- list("R", "Programming")

# Create main list with names
mylist <- list(
  vector = v,
  matrix = m,
  list_data = l
)

# Print the list
print(mylist)

# Access first element
print(mylist[[1]])

# Access third element
print(mylist[[3]])

# Access second element using name
print(mylist$matrix)

# Access both 1st and 2nd elements
print(mylist[c(1,2)])

#b.4 Create a list of 3 vectors and one numeric value
#Question
#Create a list of 3 vectors and one numeric value
#Print the list
#Print its 2nd element
#Print its 2nd and 4th element
#Print the content of 1st member of vector which as a whole is 2nd element of the list
#x[[2]][1]
```

```
#Update this content
#x[[2]][1] = 12 and print x[[2]]

# Create vectors
v1 <- c(1,2,3)
v2 <- c(4,5,6)
v3 <- c(7,8,9)

# Numeric value
num <- 100

# Create list
x <- list(v1, v2, v3, num)

# Print the list
print(x)

# Print 2nd element
print(x[[2]])

# Print 2nd and 4th element
print(x[c(2,4)])

# Print 1st member of vector which is 2nd element
print(x[[2]][1])

# Update this value
x[[2]][1] <- 12

# Print updated vector
print(x[[2]])
```

#b.5 Write an R program to find all elements of a given list that are not in another given list

```
# Create two lists
list1 <- list(1,2,3,4,5)
list2 <- list(3,4,5)

# Convert lists to vectors
v1 <- unlist(list1)
v2 <- unlist(list2)

# Find elements not present in list2
result <- setdiff(v1, v2)

# Print result
print(result)
```

#Write an R program to find row and column index of maximum and minimum value in a given matrix.

```
# Create a matrix
m <- matrix(c(10,25,15,40,5,30,12,18,50), nrow=3, byrow=TRUE)

# Print matrix
print(m)

# Find position of maximum value
max_pos <- which(m == max(m), arr.ind = TRUE)
```

Find position of minimum value : arr.ind shows the position of row and col. which shows true value index. max(m) =50, m==max(m) shows 50 ke equal konse element h.

```
min_pos <- which(m == min(m), arr.ind = TRUE)

# Print results cat() means concatenate and print.
cat("Maximum value position (row, column):\n")
```

```
print(max_pos)

cat("Minimum value position (row, column):\n")
print(min_pos)

#Use R to create the following two matrices and do the indicated matrix multiplication
# Create first matrix
A <- matrix(c(7,9,12,2,4,13),nrow=2,byrow=TRUE)

# Create second matrix
B <- matrix(c(1,7,12,19,2,8,13,20,3,9,14,21),nrow=3,byrow=TRUE)

# Print matrices
print(A)
print(B)

# Matrix multiplication
result <- A %*% B

# Print result
print(result)


#Write a R script to print Hello World 4 times using repeat loop provided the count is set at 2 initially.
v =c("hello")
count =2 #start

repeat {
  print(v)
  count = count + 1

  if(count > 5) {
```

```
    break
  }
}

#Write a R script to print Hello using while loop 5 times provided the count is set at 2 initially.
v <- "hello"
count <- 1

while(count < 7) {
  print(v)
  count <- count + 1
}

#Write a R script to print first 15 alphabets using for loop.
alphabets <- letters[1:15]

for(i in alphabets) {
  print(i)
}
#Using sum() function
sum(1:100)
n <- 100
sum <- n*(n+1)/2
print(sum)

#Using for loop with if condition
for(i in letters[1:6]) {
  if(i == "d") {
    next    # skip the letter "d"
  }
  print(i)
}
```

```
#wap to print first 8 alphabet using for loop in r
for(i in 1:8) {
  print(LETTERS[i])
}
#wap in r to print first 10 letters except f
for(i in 1:10) {
  if(letters[i] != "f") {
    print(letters[i])
  }
}
#wap in r to the square of first 10 numbers
numbers <- 1:10
squares <- numbers^2
print(squares)
#wap in r to the square of first 10 numbers using functions
printsquare=function(a)
  for(i in 1:a ){
    cat("square of numbers",i,"=",i**2,"\n")
  }
printsquare(10)
# Create a sequence of numbers from 32 to 44
a <- seq(32, 44)
print(a)

# Find mean of numbers from 25 to 82
mean_val <- mean(25:82)
print(mean_val)

# Find sum of numbers from 41 to 68
sum_val <- sum(41:68)
```

```
print(sum_val)
# Function that takes an argument n
s <- function(n) {
  for(i in 1:n) {
    print(i^2)
  }
}
s(6)
# Function without argument (fixed to 5 numbers)
sq <- function() {
  for(i in 1:5) {
    print(i^2)
  }
}
sq()
#Write a program using for loop to create a multiplication table of any number
num <- 7
for(i in 1:10) {
  print(paste(num, "x", i, "=", num * i))
}

#Write a program to count the number of even numbers in a vector
vec <- c(1,2,3,4,5,6,7,8,9,10)
even_count <- sum(vec %% 2 == 0)
print(even_count)

#WAP to print the square root of the integers one to ten
for(i in 1:10) {
  print(sqrt(i))
}

#Create a function to calculate raised power of any number
```

```
power_func <- function(base, exp) {
  return(base ^ exp)
}
print(power_func(2,3))

#Create a function fahrenheit_to_celsius that converts temperatures from Fahrenheit to
Celsius
fahrenheit_to_celsius <- function(f) {
  return((f - 32) * 5/9)
}
print(fahrenheit_to_celsius(98.6))

#Create a function that converts temperature from celsius to Kelvin
celsius_to_kelvin <- function(c) {
  return(c + 273.15)
}
print(celsius_to_kelvin(25))

#Create a function to compute area of a circle
area_circle <- function(radius) {
  return(pi * radius^2)
}
print(area_circle(5))



df = data.frame(
  name = c("A","B"),
  age = c(20,21)
)

a = array(1:6, dim = c(2,3))
```

#Q1. Implement a for loop to classify Sepal.Length in the iris dataset as "greater than 5" or "lesser than 5"

```
# Load iris dataset (already available in R)
data(iris)

# Create empty vector to store result
result <- c()

# For loop to classify values
for(i in 1:length(iris$Sepal.Length)){
  if(iris$Sepal.Length[i] > 5){
    result[i] <- "greater than 5"
  } else {
    result[i] <- "lesser than 5"
  }
}

# Print result
result
```

#Q2. Use seq() and rep() to generate sequences

```
# Using seq()
seq(1, 10, by = 2)      # 1 3 5 7 9

# Using rep()
rep(1:3, times = 2)     # 1 2 3 1 2 3

rep(1:3, each = 2)      # 1 1 2 2 3 3
```

#Q3. Create a vector and compute mean

```
# Create vector
x <- c(12,7,3,4.2,18,2,54,-21,8,-5)
```

```
# a. Mean of the vector
mean(x)

#Q3(b). Mean after trimming 3 values from each end
mean(x, trim = 3/length(x))

#Q3(c). Mean with NA values
x2 <- c(12,7,3,4.2,18,2,54,-21,8,-5,NA)

# Mean ignoring NA
mean(x2, na.rm = TRUE)

#Q. Create the vectors with following values and compute median of the vector
#(a) Vector: (12,7,3,4.2,18,2,54,-21,8,-5)
# Create vector
x <- c(12,7,3,4.2,18,2,54,-21,8,-5)

# Compute median
median(x)

#(b) Vector: (2,13,8,7,15,3,9)
# Create vector
y <- c(2,13,8,7,15,3,9)

# Compute median
median(y)

#Q. Create a vector (2,1,2,3,1,2,3,4,1,5,5,3,2,3) and compute mode of the vector
# Create vector
# Create vector
x <- c(2,1,2,3,1,2,3,4,1,5,5,3,2,3)
```

```
# Find mode
names(sort(table(x), decreasing = TRUE))[1]

#15 students were enrolled in a mathematics course.
#Their ages were (18,18,20,19,22,21,22,23,22,20,19,20,19,22,18).
#A) Find the mean age, median age and mode age of all the students.
#B) Find the median age of all students under age 22. Hint: age1=age[age<22].
#C) Two more students enter in the class, age of both the students is 23. What will be the mean age, median age and mode age now?

# Create vector
age <- c(18,18,20,19,22,21,22,23,22,20,19,20,19,22,18)

# A) Mean, Median, Mode
mean(age)
median(age)

# Mode
mode_age <- as.numeric(names(sort(table(age), decreasing = TRUE))[1])
mode_age

#b
age1 <- age[age < 22]

median(age1)

#c
# Add new students
age_new <- c(age, 23, 23)

# Mean, Median, Mode
mean(age_new)
median(age_new)
```

```
# Mode
mode_new <- as.numeric(names(sort(table(age_new), decreasing = TRUE))[1])
mode_new

#Q1. Use typeof(), length(), and class() on various data types
# Numeric
x <- 10
typeof(x)
length(x)
class(x)

# Character
y <- "Hello"
typeof(y)
length(y)
class(y)

# Vector
z <- c(1,2,3,4)
typeof(z)
length(z)
class(z)

# Logical
a <- TRUE
typeof(a)
length(a)
class(a)

#Q2. Basic plotting using plot() and hist()
# Plot
```

```
x <- c(1,2,3,4,5)
y <- c(2,4,6,8,10)

plot(x, y)

# Histogram
data <- c(10,20,20,30,40,40,40,50)
hist(data)

#Q3. Recursive functions (Factorial & Fibonacci)
# Factorial
fact <- function(n){
  if(n == 0){
    return(1)
  } else {
    return(n * fact(n-1))
  }
}

fact(5)

# Fibonacci
fib <- function(n){
  if(n <= 1){
    return(n)
  } else {
    return(fib(n-1) + fib(n-2))
  }
}

fib(6)
```

```
#Q4. Create a list and perform slicing
# Create list
my_list <- list(name="Isha", age=21, marks=c(80,85,90))

# Access elements
my_list$name
my_list[[2]]

# Slicing
my_list[1:2]

#Q1. Create a data frame and perform operations
# Create data frame
df <- data.frame(
  name = c("A","B","C"),
  age = c(20,21,22)
)

# Add column
df$marks <- c(80,85,90)

# Remove column
df$age <- NULL

# Add row
df[4,] <- c("D", 95)

# Access elements
df$name      # column
df[1,]       # row
df[1,1]      # specific value
```

```
#Q2. Expand a data frame
# Add new rows
df <- rbind(df, data.frame(name="E", marks=88))

# Add new column
df$grade <- c("A","A","B","A","B")

#Q3. Merge data frames from iris dataset
data(iris)

Df1 <- iris[1:50, ]
Df2 <- iris[51:100, ]

# Merge (combine rows)
merged_df <- rbind(Df1, Df2)

#Q4. Use apply(), lapply(), sapply()
# apply()
apply(iris[,1:4], 2, mean)

# lapply()
lapply(iris[,1:4], mean)

# sapply()
sapply(iris[,1:4], mean)

#Q5. Plot a single point (3,5)
plot(3, 5)

#Write an R program to plot multiple points using the vectors: x=(1,2,3,4,5),y=(2,4,6,8,10)
# Create vectors
x <- c(1,2,3,4,5)
```

```
y <- c(2,4,6,8,10)

# Plot points
plot(x, y)
```

#Write an R program to plot points with different colors and symbols.

```
# Create vectors
x <- c(1,2,3,4,5)
y <- c(2,4,6,8,10)

# Plot with color and symbol
plot(x, y, col="blue", pch=16)
```

#Plot the points (2,4),(4,6),(6,8),(8,10)(2,4), (4,6), (6,8), (8,10)(2,4),(4,6),(6,8),(8,10) and add appropriate axis labels and title.

```
# Points
x <- c(2,4,6,8)
y <- c(4,6,8,10)

# Plot
plot(x, y,
     xlab="X-axis",
     ylab="Y-axis",
     main="Plot of Given Points")
```

#Write an R program to draw a straight line passing through the points: (1,2),(2,4),(3,6),(4,8)(1,2), (2,4), (3,6), (4,8)(1,2),(2,4),(3,6),(4,8)

```
# Points
x <- c(1,2,3,4)
y <- c(2,4,6,8)

# Plot line
plot(x, y, type="l")
```

```
#Write an R program to plot a line graph using the vectors: x=(1,2,3,4,5)y=(5,4,3,2,1)
# Vectors
x <- c(1,2,3,4,5)
y <- c(5,4,3,2,1)

# Line graph
plot(x, y, type="l")

#Plot a line graph representing the equation: y=2x+1 where x is 1 to 10.
# Values
x <- 1:10
y <- 2*x + 1

# Plot
plot(x, y, type="l")

#Write an R program to draw a line with different line types and colors.
# Data
x <- 1:5
y <- c(2,4,6,8,10)

# Plot with different line type and color
plot(x, y, type="l", col="red", lty=2)

#Plot the line graph and add grid lines to the plot.
# Data
x <- 1:5
y <- c(5,4,3,2,1)

# Plot
plot(x, y, type="l")
```

```
# Add grid
grid()

#Q3. Recursive functions (Factorial & Fibonacci)
# Factorial
fact <- function(n){
  if(n == 0){
    return(1)
  } else {
    return(n * fact(n-1))
  }
}

fact(5)

# Fibonacci
fib <- function(n){
  if(n <= 1){
    return(n)
  } else {
    return(fib(n-1) + fib(n-2))
  }
}

fib(6)

#Create a list using list() and perform list slicing
# Create list
my_list <- list(name="Isha", age=21, marks=c(80,85,90))

# Access elements
```

```
my_list$name
my_list[[2]]

# Slicing
my_list[1:2]
```

#Create a data frame and perform various operations (add/remove columns, rows, access elements)

```
# Create data frame
df <- data.frame(
  name=c("A","B","C"),
  age=c(20,21,22)
)

# Add column
df$marks <- c(80,85,90)

# Remove column
df$age <- NULL

# Add row
df[4,] <- c("D",95)

# Access
df$name
df[1,]
df[1,1]
```

#Write a program to expand a data frame in R Perform merge operations on data frames Df1 and Df2 from the iris dataset

```
# Add row
df <- rbind(df, data.frame(name="E", marks=88))
```

```
# Add column
df$grade <- c("A","A","B","A","B")

#Use apply(), lapply(), sapply() with lists and data frames
# apply
apply(iris[,1:4], 2, mean)

# lapply
lapply(iris[,1:4], mean)

# sapply
sapply(iris[,1:4], mean)

#Create a function to summarize each column in a data frame
summary_fun <- function(df){
  sapply(df, summary)
}

summary_fun(iris)

#Use rbind() and cbind() to expand data
# rbind (add rows)
df1 <- data.frame(x=c(1,2), y=c(3,4))
df2 <- data.frame(x=c(5,6), y=c(7,8))

rbind(df1, df2)

# cbind (add columns)
a <- c(1,2,3)
b <- c(4,5,6)

cbind(a, b)
```

#Q1. productA <- c(20, 35, 40, 55, 70, 90)

#productB <- c(15, 25, 30, 45, 60, 80)

#months <- 1:6

#1. Plot both products' sales on the same graph using different colors and line types.

#2. Add appropriate axis labels and a title.

#3. Insert a legend in the top-left corner that clearly distinguishes between Product A and Product B

```
# Data
productA <- c(20, 35, 40, 55, 70, 90)
productB <- c(15, 25, 30, 45, 60, 80)
months <- 1:6

# Plot Product A
plot(months, productA, type="l", col="blue", lty=1,
    xlab="Months", ylab="Sales",
    main="Sales of Product A and Product B")

# Add Product B
lines(months, productB, col="red", lty=2)

# Add legend
legend("topleft",
      legend=c("Product A", "Product B"),
      col=c("blue","red"),
      lty=c(1,2))
```

#Q2. You are given the following data representing the height (in cm) and weight (in kg) of 10 students:

#height <- c(150, 152, 155, 160, 162, 165, 168, 170, 172, 175)

#weight <- c(45, 47, 50, 55, 58, 60, 62, 65, 68, 70)

#1. Create a scatter plot of height vs. weight.

```
# Data
height <- c(150, 152, 155, 160, 162, 165, 168, 170, 172, 175)
weight <- c(45, 47, 50, 55, 58, 60, 62, 65, 68, 70)

# Scatter plot
plot(height, weight,
     xlab="Height (cm)",
     ylab="Weight (kg)",
     main="Height vs Weight",
     col="blue",
     pch=16)
```

#2. Use different point symbols and colors to make the plot visually clear.

#3. Add appropriate axis labels and a title.

#4. Insert a legend that explains the meaning of the chosen point symbols/colors (e.g., male vs female students, or group A vs group B).

```
# Data
height <- c(150, 152, 155, 160, 162, 165, 168, 170, 172, 175)
weight <- c(45, 47, 50, 55, 58, 60, 62, 65, 68, 70)

# Create groups (example: first 5 = Group A, next 5 = Group B)
group <- c(rep("A",5), rep("B",5))

# Assign colors and symbols
col_vec <- ifelse(group=="A", "blue", "red")
pch_vec <- ifelse(group=="A", 16, 17)

# Scatter plot
plot(height, weight,
     col=col_vec,
     pch=pch_vec,
```

```
    xlab="Height (cm)",
    ylab="Weight (kg)",
    main="Height vs Weight (Group A & B)")

# Add legend
legend("topleft",
      legend=c("Group A","Group B"),
      col=c("blue","red"),
      pch=c(16,17))

#Q3. A company records the quarterly sales distribution (in percentage) across four regions:
#sales <- c(25, 30, 20, 25)
#regions <- c("North", "South", "East", "West")
#Write R code to:
#  1.    Create a pie chart showing the sales distribution.
#2.      Add labels for each region.
#3.      Use different colors for each slice.
#4.      Add a legend that explains which color corresponds to which region.

# Data
sales <- c(25, 30, 20, 25)
regions <- c("North", "South", "East", "West")

# Colors
colors <- c("blue", "red", "green", "yellow")

# Pie chart
pie(sales,
   labels=regions,
   col=colors,
   main="Sales Distribution by Region")
```

```
# Legend
legend("topright",
       legend=regions,
       fill=colors)

#Q4. You are given monthly temperature data for a city:
#months <- 1:12
#temperature <- c(15, 18, 22, 28, 32, 35, 34, 33, 30, 25, 20, 16)
#Write R code to:
#  1.    Plot the temperature as a line chart.
#2.      Add axis labels and a title. 3.3.          Use a red dashed line (lty = 2).

# Data
months <- 1:12
temperature <- c(15, 18, 22, 28, 32, 35, 34, 33, 30, 25, 20, 16)

# Line chart
plot(months, temperature,
     type="l",
     col="red",
     lty=2,
     xlab="Months",
     ylab="Temperature (°C)",
     main="Monthly Temperature Variation")

#Q5. A company records sales across four regions:
#sales <- c(120, 150, 100, 130)
#regions <- c("North", "South", "East", "West")
#Write R code to:
 # 1.    Create a bar chart of sales by region.
#2.      Use different colors for each bar.
#3.      Add a legend explaining the regions
```

```
# Data
sales <- c(120, 150, 100, 130)
regions <- c("North", "South", "East", "West")

# Colors
colors <- c("blue", "red", "green", "yellow")

# Bar chart
barplot(sales,
        names.arg=regions,
        col=colors,
        xlab="Regions",
        ylab="Sales",
        main="Sales by Region")

# Legend
legend("topright",
       legend=regions,
       fill=colors)

#Q6. You have exam scores of students from two different classes:
#classA <- c(65, 70, 75, 80, 85, 90)
#classB <- c(60, 62, 68, 72, 78, 82)
#Write R code to:
 # 1.    Create a box plot comparing Class A and Class B scores.
#2.     Use different colors for each box.
#3.     Add a title and axis labels.

# Data
classA <- c(65, 70, 75, 80, 85, 90)
classB <- c(60, 62, 68, 72, 78, 82)
```

```
# Combine data
data <- list(ClassA = classA, ClassB = classB)

# Box plot
boxplot(data,
        col=c("blue","red"),
        xlab="Classes",
        ylab="Scores",
        main="Comparison of Class A and Class B Scores")

#Q7. You are given height and weight data of 10 students:
#height <- c(150, 152, 155, 160, 162, 165, 168, 170, 172, 175)
#weight <- c(45, 47, 50, 55, 58, 60, 62, 65, 68, 70) Write R code to:
#  1.    Create a scatter plot of height vs weight.
#2.      Use different point symbols (pch) for male and female students.
#3.      Add a legend to distinguish between the two groups.

# Data
height <- c(150, 152, 155, 160, 162, 165, 168, 170, 172, 175)
weight <- c(45, 47, 50, 55, 58, 60, 62, 65, 68, 70)

# Create groups (first 5 = Male, next 5 = Female)
gender <- c(rep("Male",5), rep("Female",5))

# Assign symbols
pch_vec <- ifelse(gender=="Male", 16, 17)

# Plot
plot(height, weight,
     pch=pch_vec,
     col="blue",
     xlab="Height (cm)",
```

```
     ylab="Weight (kg)",
     main="Height vs Weight")

# Legend
legend("topleft",
       legend=c("Male","Female"),
       pch=c(16,17))

#Q8. A company’s quarterly sales distribution is:
#sales <- c(25, 30, 20, 25)
#regions <- c("North", "South", "East", "West")
#Write R code to:
 # 1.     Create a pie chart showing sales distribution.
#2.       Use different colors for each slice.
#3.       Add a legend explaining the regions.
# Data
sales <- c(25, 30, 20, 25)
regions <- c("North", "South", "East", "West")

# Colors
colors <- c("blue", "red", "green", "yellow")

# Pie chart
pie(sales,
    col=colors,
    main="Sales Distribution")

# Legend
legend("topright",
       legend=regions,
       fill=colors)
```

```
#1.      Generate Exploratory Analysis of Big Mart
#Data Set and Train Data Set

# Load dataset
data <- read.csv("Train.csv")

# View data
head(data)
str(data)
summary(data)

# Check missing values
colSums(is.na(data))

# Histogram (Sales)
hist(data$Item_Outlet_Sales,
     col="blue",
     main="Sales Distribution",
     xlab="Sales")

# Bar plot (Outlet Type)
barplot(table(data$Outlet_Type),
        col="green",
        main="Outlet Type Count")

# Boxplot (Fat Content vs Sales)
boxplot(Item_Outlet_Sales ~ Item_Fat_Content,
        data=data,
        col=c("red","yellow"),
        main="Sales vs Fat Content")

# Scatter plot (MRP vs Sales)
```

```
plot(data$Item_MRP, data$Item_Outlet_Sales,
     col="blue",
     pch=16,
     xlab="MRP",
     ylab="Sales",
     main="MRP vs Sales")

# Handle missing values (example)
data$Item_Weight[is.na(data$Item_Weight)] <-
  mean(data$Item_Weight, na.rm=TRUE)

#2.       Graphically represent Big Mart Data Set
#using:
 # -Univariate Analysis (Histogram, Boxplot)
#-Bivariate Analysis (Scatter plot, Bar plot)
# Load dataset
data <- read.csv("Train.csv")

# -------------------------------
# Univariate Analysis
# -------------------------------

# Histogram (Item Outlet Sales)
hist(data$Item_Outlet_Sales,
     col="blue",
     main="Histogram of Sales",
     xlab="Sales")

# Boxplot (Item Outlet Sales)
boxplot(data$Item_Outlet_Sales,
        col="red",
        main="Boxplot of Sales")
```

```
# -------------------------------
# Bivariate Analysis
# -------------------------------

# Scatter Plot (MRP vs Sales)
plot(data$Item_MRP, data$Item_Outlet_Sales,
     col="green",
     pch=16,
     xlab="MRP",
     ylab="Sales",
     main="MRP vs Sales")

# Bar Plot (Outlet Type vs Count)
barplot(table(data$Outlet_Type),
        col="orange",
        main="Outlet Type Count",
        xlab="Outlet Type",
        ylab="Count")

#3.      Graphically represent Train Data Set using:
#Univariate and Bivariate Analysis

# Load dataset
data <- read.csv("Train.csv")

# -------------------------
# Univariate Analysis
# -------------------------

# Histogram (Sales)
hist(data$Item_Outlet_Sales,
```

```
    col="blue",
    main="Histogram of Sales",
    xlab="Sales")

# Boxplot (Sales)
boxplot(data$Item_Outlet_Sales,
        col="red",
        main="Boxplot of Sales")

# -------------------------
# Bivariate Analysis
# -------------------------

# Scatter Plot (MRP vs Sales)
plot(data$Item_MRP, data$Item_Outlet_Sales,
    col="green",
    pch=16,
    xlab="MRP",
    ylab="Sales",
    main="MRP vs Sales")

# Bar Plot (Outlet Size)
barplot(table(data$Outlet_Size),
        col="orange",
        main="Outlet Size Count",
        xlab="Outlet Size",
        ylab="Count")

#4.      Implement a Linear Model using 100
#variables in R

# Create sample data with 100 variables
```

```
set.seed(123)

data <- data.frame(matrix(rnorm(1000), nrow=10))
colnames(data) <- paste0("X", 1:100)

# Create dependent variable
data$Y <- rnorm(10)

# Apply Linear Model
model <- lm(Y ~ ., data=data)

# View summary
summary(model)

#5.      Import/export data using read.csv(),
#write.csv(), readxl, jsonlite

1# Import CSV file
data <- read.csv("file.csv")

# View data
head(data)

#2 Export data to CSV
write.csv(data, "output.csv", row.names=FALSE)

#3 Install and load package
install.packages("readxl")
library(readxl)

# Read Excel file
data_excel <- read_excel("file.xlsx")
```

```
# 4Install and load package
install.packages("jsonlite")
library(jsonlite)

#5 Read JSON file
data_json <- fromJSON("file.json")

#6 Write JSON file
write_json(data_json, "output.json")

#6.      Perform data cleaning using na.omit(), is.na(), and tidyr package
#1. Create sample data
data <- data.frame(
  name = c("A","B","C","D"),
  marks = c(80, NA, 90, NA),
  age = c(20, 21, NA, 22)
)

data
#2. Using is.na() (check missing values)
# Check NA values
is.na(data)

# Count NA in each column
colSums(is.na(data))

#3. Using na.omit() (remove missing values)
clean_data <- na.omit(data)

clean_data
#4. Using tidyr package
```

```
# Install and load tidyr
install.packages("tidyr")
library(tidyr)

# Remove rows with NA
drop_na(data)

# Replace NA with value (example: 0)
replace_na(data, list(marks=0, age=0))
```

#7. Create a prediction model using lm() and evaluate with summary() and residual plots

```
# Create sample data
data <- data.frame(
  x = c(1,2,3,4,5),
  y = c(2,4,5,4,5)
)

# Create linear model
model <- lm(y ~ x, data=data)

# Model summary
summary(model)

# Predictions
pred <- predict(model, newdata = data.frame(x=c(6,7)))
pred

# Residual plot
plot(model$fitted.values, model$residuals,
     xlab="Fitted Values",
     ylab="Residuals",
     main="Residual Plot")
```

```
abline(h=0, col="red")
```

#8. To build a system in R that analyzes astrological charts and Panchang (Hindu almanac) data to predict doshas (planetary defects) and study correlations with health patterns or seasonal behaviors, aligning with IKS and modern data analytics.

```
# ----------------------------
# 1. Create Sample Dataset
# ----------------------------
data <- data.frame(
  date = as.Date('2024-01-01') + 0:9,
  tithi = c("Pratipada","Dwitiya","Tritiya","Chaturthi","Panchami",
            "Shashthi","Saptami","Ashtami","Navami","Dashami"),
  nakshatra = c("Ashwini","Bharani","Krittika","Rohini","Mrigashira",
                "Ardra","Punarvasu","Pushya","Ashlesha","Magha"),
  dosha = c(1,0,1,0,1,0,1,0,1,0),   # 1 = dosha present
  temperature = c(15,18,20,22,25,28,30,32,29,26),
  health_score = c(80,78,75,82,70,68,65,60,72,75)
)

# ----------------------------
# 2. Explore Data
# ----------------------------
summary(data)

# ----------------------------
# 3. Convert categorical data
# ----------------------------
data$tithi <- as.factor(data$tithi)
data$nakshatra <- as.factor(data$nakshatra)

# ----------------------------
# 4. Build Prediction Model
```

```
# ----------------------------
model <- lm(health_score ~ dosha + temperature, data=data)

# Model summary
summary(model)

# ----------------------------
# 5. Correlation Analysis
# ----------------------------
cor(data$temperature, data$health_score)

# ----------------------------
# 6. Visualization
# ----------------------------
# Scatter plot
plot(data$temperature, data$health_score,
     col="blue", pch=16,
     xlab="Temperature",
     ylab="Health Score",
     main="Temperature vs Health")

# Boxplot (Dosha vs Health)
boxplot(health_score ~ dosha, data=data,
        col=c("green","red"),
        main="Health vs Dosha")

# ----------------------------
# 7. Residual Plot
# ----------------------------
plot(model$fitted.values, model$residuals,
     xlab="Fitted",
     ylab="Residuals",
```

```
    main="Residual Plot")
abline(h=0, col="red")


#To analyze, model, and visualize air and water pollution trends using real-time
environmental datasets (e.g., AQI, water  pH, turbidity, heavy metals), and predict future
pollution levels using statistical and machine learning techniques in R.
# ----------------------------
# 1. Create Sample Dataset
# ----------------------------
data <- data.frame(
  day = 1:10,
  AQI = c(120, 135, 150, 160, 170, 180, 175, 165, 155, 145),
  pH = c(7.2, 7.1, 6.9, 6.8, 6.7, 6.5, 6.6, 6.8, 7.0, 7.1),
  turbidity = c(3, 4, 5, 6, 7, 8, 7, 6, 5, 4),
  heavy_metals = c(0.2,0.3,0.4,0.5,0.6,0.7,0.6,0.5,0.4,0.3)
)

# ----------------------------
# 2. Explore Data
# ----------------------------
summary(data)

# ----------------------------
# 3. Visualization
# ----------------------------

# AQI Trend (Air Pollution)
plot(data$day, data$AQI, type="l", col="red",
     xlab="Day", ylab="AQI",
     main="Air Pollution Trend")

# Water Quality (pH vs Turbidity)
plot(data$pH, data$turbidity,
```

```
     col="blue", pch=16,
     xlab="pH", ylab="Turbidity",
     main="Water Quality Analysis")

# ----------------------------
# 4. Correlation Analysis
# ----------------------------
cor(data[,2:5])

# ----------------------------
# 5. Linear Model (Prediction)
# Predict AQI based on other factors
model <- lm(AQI ~ pH + turbidity + heavy_metals, data=data)

# Model summary
summary(model)

# ----------------------------
# 6. Future Prediction
# ----------------------------
new_data <- data.frame(
  pH = c(6.5, 6.4),
  turbidity = c(8, 9),
  heavy_metals = c(0.7, 0.8)
)

predict(model, newdata=new_data)

# ----------------------------
# 7. Residual Plot
# ----------------------------
plot(model$fitted.values, model$residuals,
```

```
    xlab="Fitted Values",
    ylab="Residuals",
    main="Model Evaluation")
abline(h=0, col="blue")
```

#To analyze and predict stock price movements using historical market data and optimize investment portfolios using advanced statistical and machine learning techniques in R

```
# ----------------------------
# 1. Load Required Libraries
# ----------------------------
install.packages("quantmod")
install.packages("PerformanceAnalytics")

library(quantmod)
library(PerformanceAnalytics)

# ----------------------------
# 2. Load Historical Stock Data
# ----------------------------
# Example: Apple stock
getSymbols("AAPL", src="yahoo", from="2023-01-01")

# View data
head(AAPL)

# ----------------------------
# 3. Plot Stock Price
# ----------------------------
chartSeries(AAPL, name="Apple Stock Price")

# ----------------------------
# 4. Calculate Returns
```

```
# ----------------------------
returns <- dailyReturn(Cl(AAPL))

# Plot returns
plot(returns, main="Daily Returns")

# ----------------------------
# 5. Build Prediction Model
# ----------------------------
# Convert to data frame
data <- data.frame(
  price = as.numeric(Cl(AAPL)),
  lag1 = lag(as.numeric(Cl(AAPL)), 1)
)

# Remove NA
data <- na.omit(data)

# Linear Model
model <- lm(price ~ lag1, data=data)

# Summary
summary(model)

# ----------------------------
# 6. Predict Future Price
# ----------------------------
new_data <- data.frame(lag1 = tail(data$price,1))
predict(model, newdata=new_data)

# ----------------------------
# 7. Portfolio Optimization
```

```
# ----------------------------
# Load multiple stocks
getSymbols(c("AAPL","MSFT","GOOG"), src="yahoo", from="2023-01-01")

# Combine returns
returns_port <- na.omit(merge(
  dailyReturn(Cl(AAPL)),
  dailyReturn(Cl(MSFT)),
  dailyReturn(Cl(GOOG))
))

# Portfolio performance
charts.PerformanceSummary(returns_port)
```

#To use statistical analysis and machinelearning in R to explore, model, andvisualize gender equality across education,employment, income, and political participation, drawing from global datasets

```
# ----------------------------
# 1. Create Sample Dataset
# ----------------------------
data <- data.frame(
  education = c(60,65,70,75,80,85,90,95,85,75),   # % female education
  employment = c(50,55,60,65,70,75,80,82,78,72), # % female employment
  income = c(2000,2200,2500,2800,3000,3200,3500,3700,3400,3100),
  political = c(10,12,15,18,20,22,25,27,24,21),   # % representation
  gender_index = c(0.5,0.55,0.6,0.65,0.7,0.75,0.8,0.85,0.78,0.72)
)

# ----------------------------
# 2. Explore Data
# ----------------------------
summary(data)
```

```
# ----------------------------
# 3. Correlation Analysis
# ----------------------------
cor(data)

# ----------------------------
# 4. Visualization
# ----------------------------

# Scatter plot (Education vs Gender Index)
plot(data$education, data$gender_index,
     col="blue", pch=16,
     xlab="Education (%)",
     ylab="Gender Equality Index",
     main="Education vs Gender Equality")

# Bar plot (Political Participation)
barplot(data$political,
        col="green",
        main="Political Participation")

# ----------------------------
# 5. Build Machine Learning Model
# ----------------------------
model <- lm(gender_index ~ education + employment + income + political, data=data)

# Model summary
summary(model)

# ----------------------------
# 6. Prediction
# ----------------------------
```

```
new_data <- data.frame(
  education=88,
  employment=78,
  income=3300,
  political=23
)

predict(model, newdata=new_data)

# ----------------------------
# 7. Residual Plot
# ----------------------------
plot(model$fitted.values, model$residuals,
     xlab="Fitted Values",
     ylab="Residuals",
     main="Model Evaluation")
abline(h=0, col="red")
```